



A CSR initiative by



kotak
Kotak Mahindra Bank

Kotak IISc AI-ML Centre



TANUH

Translational Artificial Intelligence
for Networked Universal Healthcare

Deep Foundation Models

Lecture Series - Part 3

Sriram Ganapathy

Electrical Engineering, IISc.

Scope

- ✓ In this lecture series - what is covered
 - * The question of “what” in the topics listed
 - * A part of the “why” as well
 - * And a small part of “where” it is used
- ✓ In this lecture series - what is not covered
 - * Mathematical derivations
 - * Details of various methods, analysis and understanding
 - * Alternate approaches (I provide only a biased sampling)
 - * Implementation aspects

What have covered thus far

Transformers
+
Self-supervision

Lecture-1

LLMs
Pre-training and
Post-training

Lecture-2

Test-time scaling,
compression,
Diffusion, Benchmarking

Lecture-3*

Contents - This Lecture

- ✓ 3.1 Large language models - test-time scaling
 - * Chain of Thought, Tree of thought, RAG
- ✓ 3.2 Model Compression
- ✓ 3.3 Diffusion
 - * Stable Diffusion
- ✓ 3.4 Benchmarking of LLMs
- ✓ 3.5 Summary

Contents - This Lecture

- ✓ 3.1 Large language models - test-time scaling
 - * Chain of Thought, Tree of thought, RAG
- ✓ 3.2 Model Compression
- ✓ 3.3 Diffusion
 - * Stable Diffusion
- ✓ 3.4 Benchmarking of LLMs
- ✓ 3.5 Summary

3.1 LLM post-training - Test-time Scaling

✓ Test-time scaling -

Keep the model fixed, but spend more compute at inference (more tokens, more samples, more search) to get better answers.

✓ Definition of a “thought”

- * a partial plan
- * a sub-goal
- * an intermediate calculation
- * a candidate next move (in puzzles/games)
- * a partial proof or sketch

Chain of Thought

Chain-of-Thought Prompting Elicits Reasoning in Large Language Models

Jason Wei

Xuezhi Wang

Dale Schuurmans

Maarten Bosma

Brian Ichter

Fei Xia

Ed H. Chi

Quoc V. Le

Denny Zhou

Google Research, Brain Team
{jasonwei,dennyzhou}@google.com

Chain of Thought - Big Picture

Standard Prompting

Model Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

A: The answer is 27. ❌

Chain-of-Thought Prompting

Model Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: Roger started with 5 balls. 2 cans of 3 tennis balls each is 6 tennis balls. $5 + 6 = 11$. The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

A: The cafeteria had 23 apples originally. They used 20 to make lunch. So they had $23 - 20 = 3$. They bought 6 more apples, so they have $3 + 6 = 9$. The answer is 9. ✅

Chain of Thought

- ✓ Formally, let $(x \rightarrow y)$ denote the input to output required from the LLM.
- ✓ CoT turns this into a multi-step mapping
 - $x \rightarrow (r_1, r_2, \dots, r_K) \rightarrow y$
 - Where the intermediate reasoning steps r_t (“the chain of thought”).
- ✓ In practice, you do this just by prompting:
 - *Standard prompt:
“Q: If you have 3 apples and get 2 more, how many apples do you have? A:”
 - *CoT prompt:
“Q: ... A: Let’s think step by step.”

Chain of Thought

✓ Why does it work

* Standard mapping $x \rightarrow y$, the transformer stack is stuck in a tiny-class of realizable functions.

* Chain of thought mapping

● Every step, $r_{k+1} = f(x, r_1, \dots, r_k)$

● Increases the set of realizable functions polynomially through incremental decoding steps.

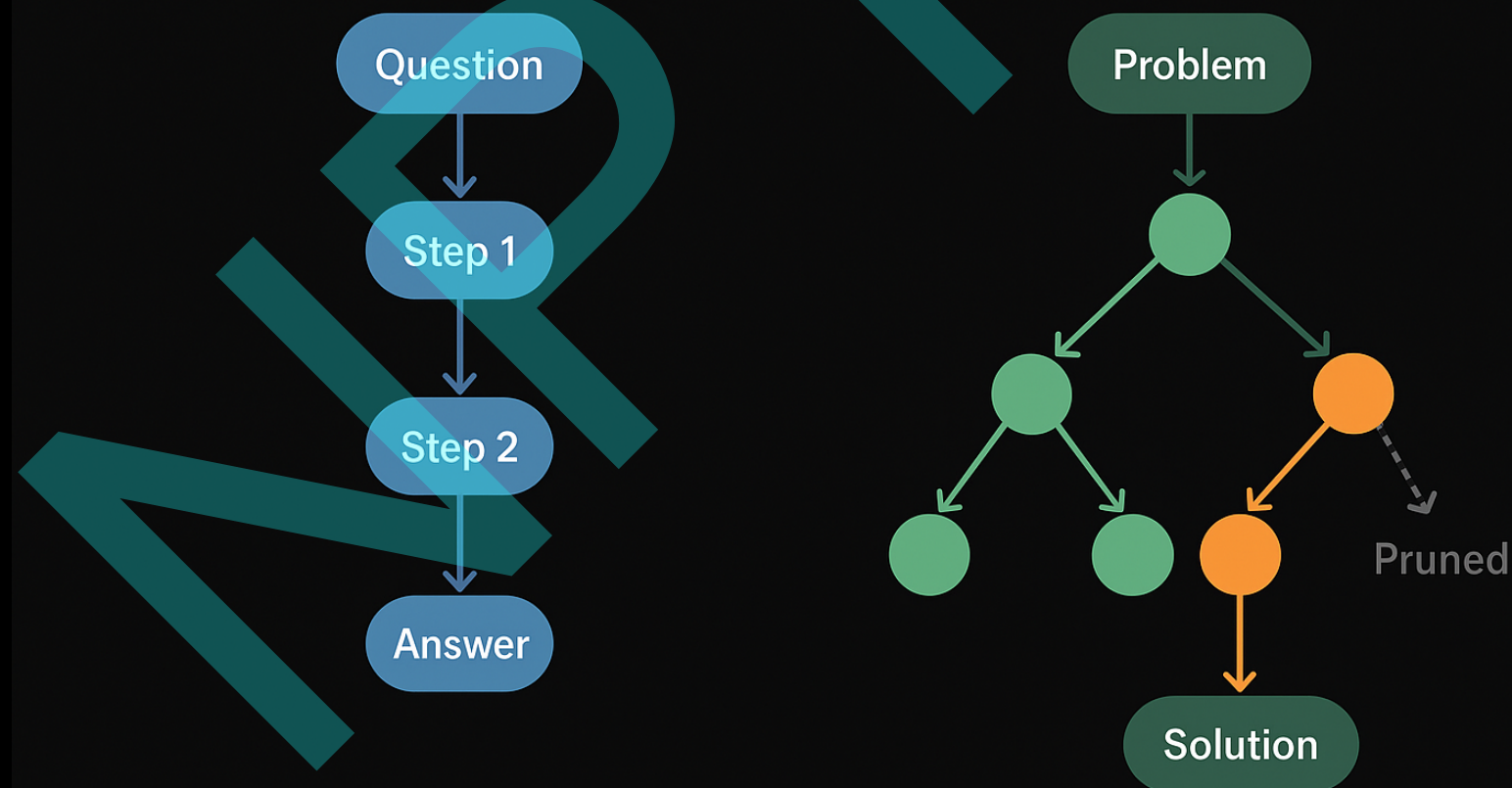
* Also, increases the interpretability of the results.

* Widely used.

Tree of thought

- ✓ Tree-of-Thought (ToT) prompting is a strategy where you make a language model **explore multiple reasoning paths like a search tree**, instead of following a single linear chain of thought.

Chain-of-Thought vs Tree-of-Thought Prompting



ToT versus CoT

Aspect	Chain-of-Thought (CoT)	Tree-of-Thought (ToT)
Structure	Single linear sequence of thoughts	Branching tree of thoughts
Exploration	Greedy / one-path	Multiple candidate paths explored
Use of model	Generator only	Generator and evaluator
Search algorithms	Implicit (via decoding)	Explicit (DFS, BFS, beam search, Monte Carlo style)
Robustness	Sensitive to early errors	Can recover by following alternative branches
Cost	Cheaper (one pass)	More expensive (many calls, but often higher success)

Test-time scaling - RAG

- ✓ The ideology - “Let us ponder more, find more relevant information before decision making”.
- ✓ You have two parts:
 - * A **retriever** that finds relevant documents/passages from a corpus
 - * A **generator** (LLM) that answers using those documents as extra context

Test-time scaling - RAG

- ✓ Input: query x (question, prompt, etc.)
- ✓ Corpus: $\mathcal{D} = \{d_1, \dots, d_N\}$
- ✓ Retriever picks a small subset of documents $z = (d_{i_1}, \dots, d_{i_k})$ from $\mathcal{D}$
- ✓ Generator produces answer y conditioned on x, z
- ✓ Conceptually, $y = \text{Generator}(x, \text{Retriever}(x, \mathcal{D}))$
 - * The generator has access to “relevant” documents in addition to the prompt to generate the response
 - ◎ This allows a refresh of the memory or to incorporate new knowledge in prediction.

Test-time scaling - RAG

- ✓ Mathematically, let the retriever be denoted $z \sim p_{\theta}(z \mid x, \mathcal{D})$
 - ✓ Encode the query and documents $h_q = f_{\theta}(x), \quad h_i = g_{\theta}(d_i)$
 - ✓ Compute similarity and find top-K similar documents
 - * $s(x, d_i) = h_q^{\top} h_i$
 - * $z^{\star} = \text{TopK}_i s(x, d_i)$
 - * Total loss function (end-to-end loss)
$$\mathcal{L}(\theta, \phi) = - \sum_{(x,y)} \log p(y \mid x, \mathcal{D}) = - \sum_{(x,y)} \log \sum_z p_{\phi}(y \mid x, z) p_{\theta}(z \mid x, \mathcal{D})$$
- Done in a 2-stage optimization in practice

RAG — Why does it work

✓ Why does that help?

* LLMs are good at in-context learning

* By retrieving similar documents/examples and concatenating them with the query, we are:

● Letting the LLM do a kind of local adaptation without changing weights.

✓ Mathematically, $p_{\text{RAG}}(y | x, \mathcal{D}) = \sum_z p_{\phi}(y | x, z) p_{\theta}(z | x, \mathcal{D})$

✓ If the retrieved information is relevant, then $H(Y | X, Z) \leq H(Y | X)$

* Uncertainty of the outputs reduce with RAG.

Contents - This Lecture

- ✓ 3.1 Large language models - test-time scaling
 - * Chain of Thought, Tree of thought, RAG
- ✓ 3.2 Model Compression
- ✓ 3.3 Diffusion
 - * Stable Diffusion
- ✓ 3.4 Benchmarking of LLMs
- ✓ 3.5 Summary

3.2 Model compression techniques

✓ Large language models are ... “large” ...

* Need for reducing model-sizes

● Reducing memory foot-print

● Latency and throughput

● Cost

● Edge / on-device deployment

Model compression techniques

✓ Simple model compression

* Quantizing the model weights post-training

● INT8 / INT4, GPTQ, ...

- Pros

- Big memory reduction, faster matmuls

- Cons

- Careful calibration needed; very low bits can hurt quality

Model compression techniques

✓ Model pruning

- * Magnitude pruning, head/block pruning

- Individual weights, heads, neurons

- * Pros

- Fewer parameters & FLOPs

- * Cons

- Unstructured sparsity

- Difficult to judge the impact upfront

Model compression techniques

✓ Low-rank / Factorization

- * LoRA, tensor/low-rank decompositions
- * Reducing effective rank of weight matrices

✓ Pros

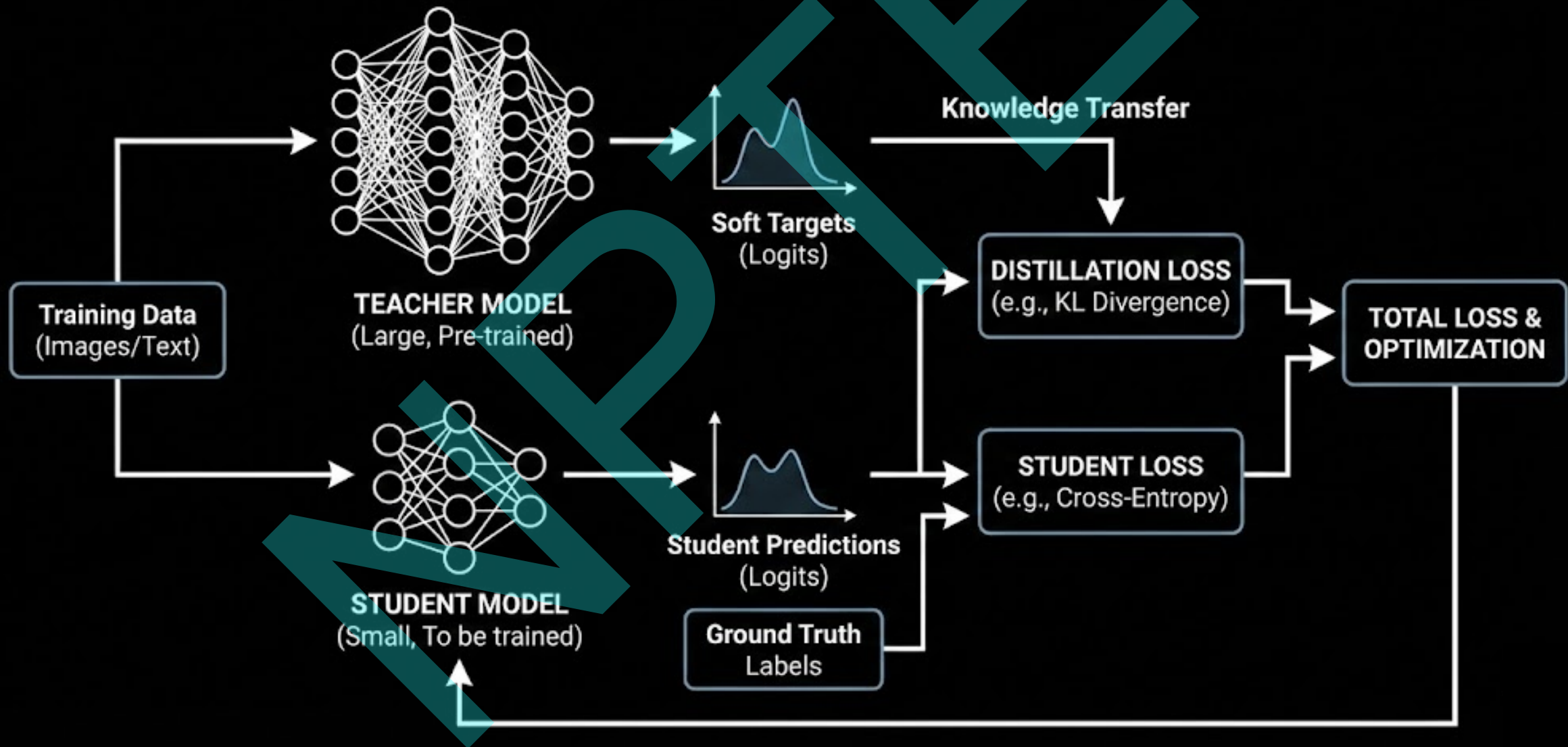
- * Smaller effective model, cheap adapters

✓ Cons

- * May limit capacity if rank too small
- * extra decomposition step

Model compression techniques

TEACHER-STUDENT KNOWLEDGE DISTILLATION



Model compression techniques

- ✓ Knowledge Distillation — (Most effective)
- ✓ Teacher–student distillation, task distillation
 - Entire model (student smaller than teacher)

*Pros

- Much smaller model with similar behavior

*Cons

- Training student adds cost
- May lose some generality

Time for some ...

NPTFEL

Q&A

Contents - This Lecture

- ✓ 3.1 Large language models - test-time scaling
 - * Chain of Thought, Tree of thought, RAG
- ✓ 3.2 Model Compression
- ✓ 3.3 Diffusion
 - * Stable Diffusion
- ✓ 3.4 Benchmarking of LLMs
- ✓ 3.5 Summary

3.3 Diffusion models

Denoising Diffusion Probabilistic Models

Jonathan Ho
UC Berkeley

`jonathanho@berkeley.edu`

Ajay Jain
UC Berkeley

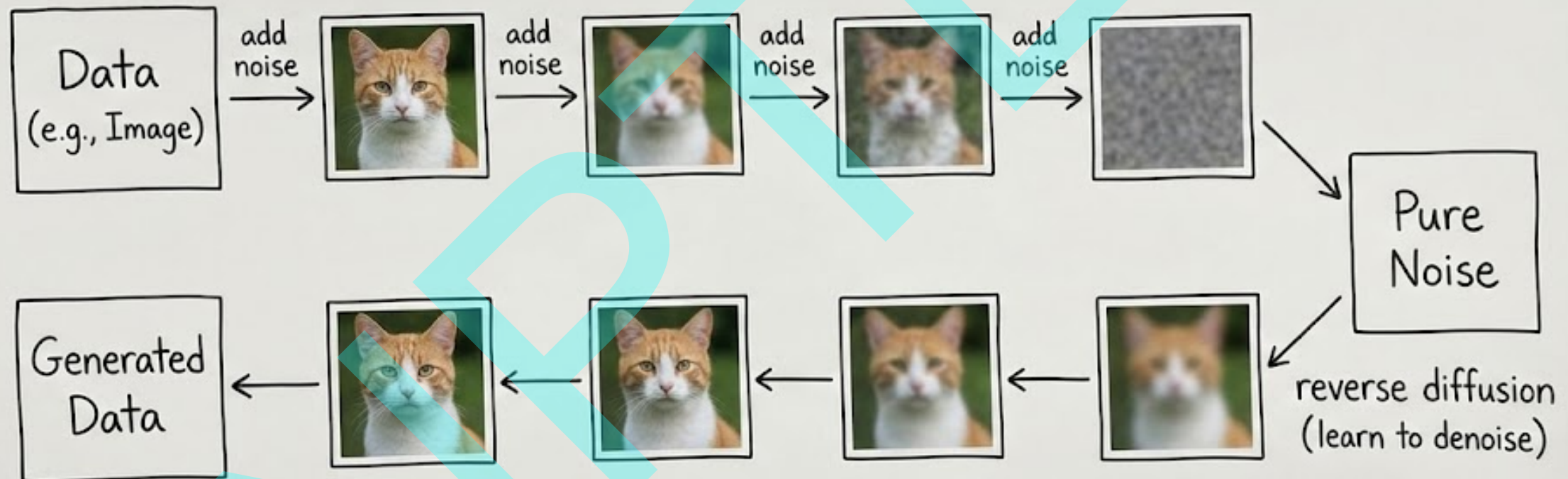
`ajayj@berkeley.edu`

Pieter Abbeel
UC Berkeley

`pabbeel@cs.berkeley.edu`

Diffusion model (Big-picture)

DIFFUSION MODELS



Forward Process: degrades data. Reverse Process: learns to restore data.

Diffusion models (forward)

- ✓ Let $\mathbf{x}_0$ denote a real data point (e.g., an image), and let $\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_T$ denote progressively noisier versions of $\mathbf{x}_0$.
- ✓ The forward process is defined as a fixed Markov chain that gradually adds Gaussian noise:

$$q(\mathbf{x}_t | \mathbf{x}_{t-1}) = \mathcal{N}(\mathbf{x}_t; \sqrt{1 - \beta_t} \mathbf{x}_{t-1}, \beta_t \mathbf{I}), \quad t = 1, \dots, T,$$

where $\beta_t \in (0, 1)$ is a variance schedule (typically small).

We define.

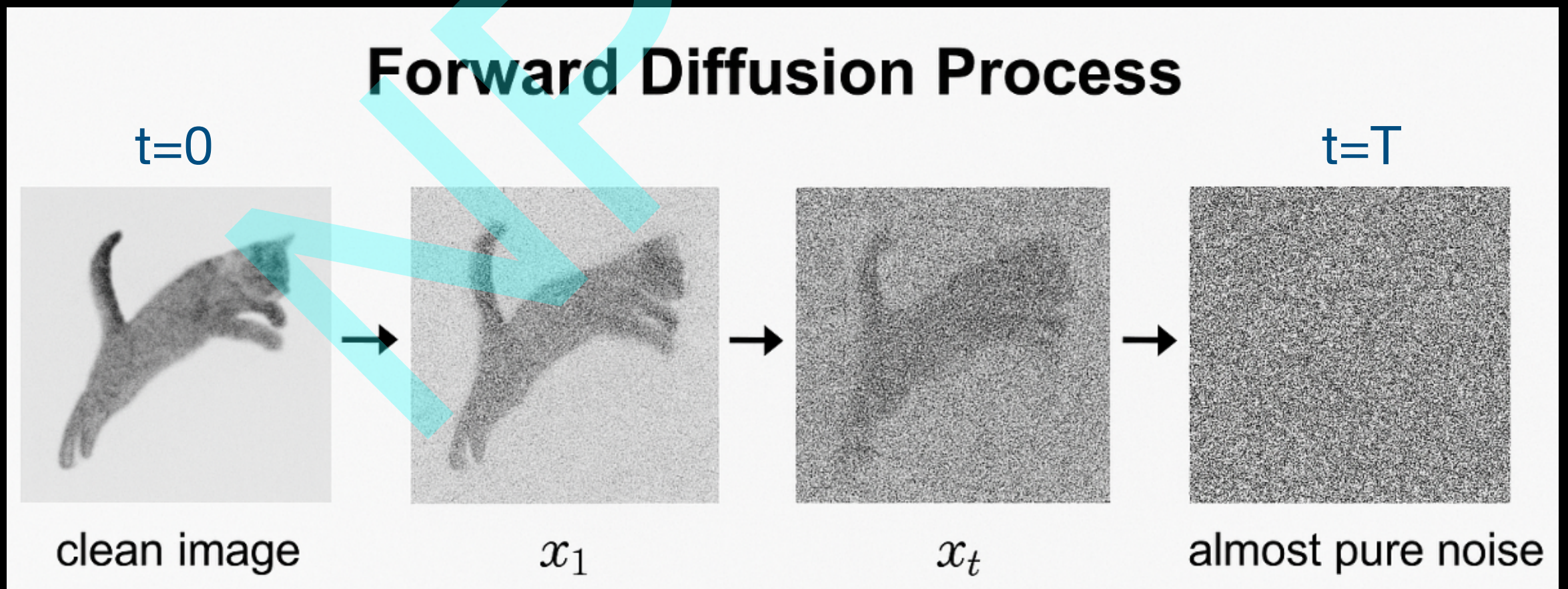
$$\alpha_t = 1 - \beta_t,$$
$$\bar{\alpha}_t = \prod_{s=1}^t \alpha_s.$$

Due to the linear-Gaussian structure, we can express the distribution

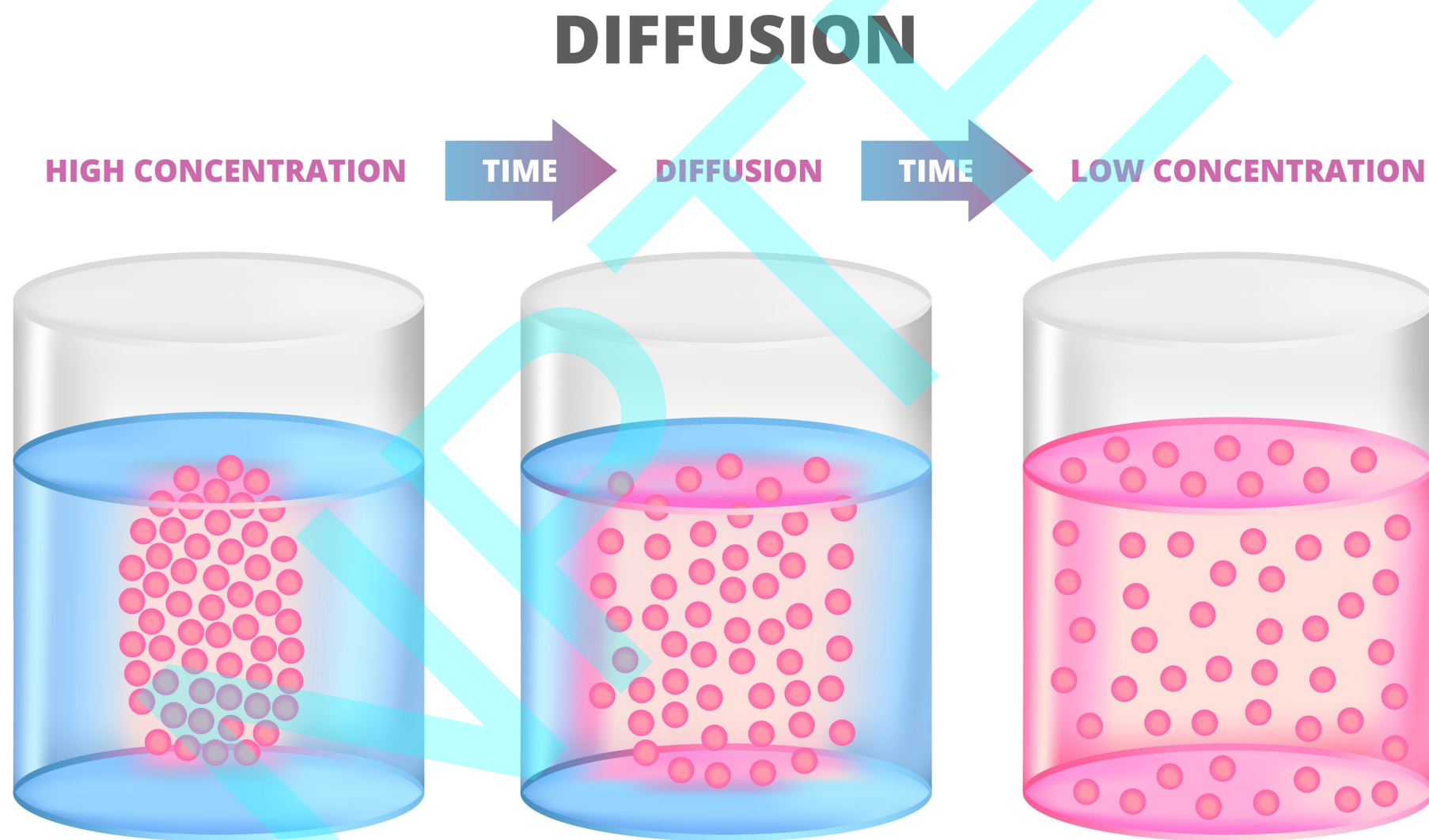
$$q(\mathbf{x}_t | \mathbf{x}_0) = \mathcal{N}(\mathbf{x}_t; \sqrt{\bar{\alpha}_t} \mathbf{x}_0, (1 - \bar{\alpha}_t) \mathbf{I}).$$

Diffusion models (forward)

- ✓ Equivalently, a sample $\mathbf{x}_t$ from this distribution can be constructed as
- ✓ $\mathbf{x}_t = \sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, \quad \epsilon \sim \mathcal{N}(0, \mathbf{I}).$
- ✓ This allows us to obtain a noisy version of any data point at any time step t with a single sampling step.



Diffusion models (forward)



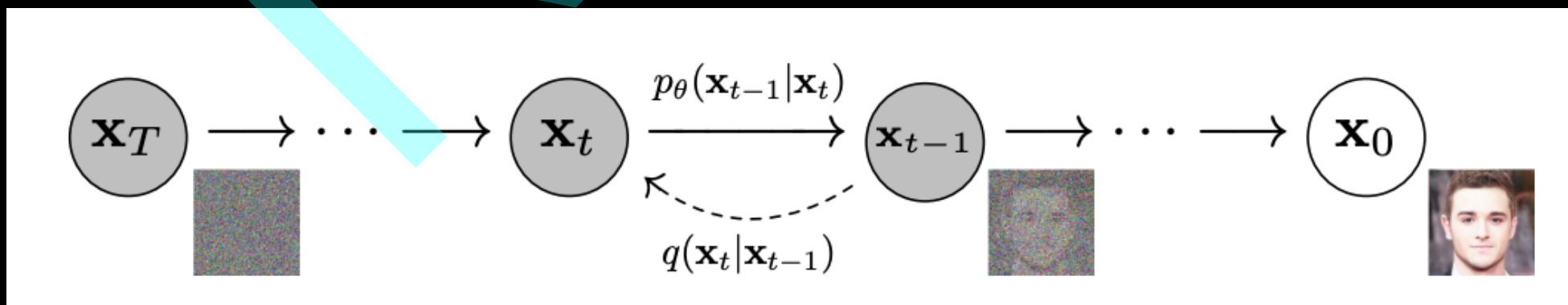
Diffusion models (backward)

- ✓ The generative process is defined as a reverse Markov chain $p_{\theta}(\mathbf{x}_{0:T})$ parameterized by θ :

$$p_{\theta}(\mathbf{x}_{0:T}) = p(\mathbf{x}_T) \prod_{t=1}^T p_{\theta}(\mathbf{x}_{t-1} | \mathbf{x}_t),$$

where the prior $p(\mathbf{x}_T)$ is chosen to be standard Gaussian, $p(\mathbf{x}_T) = \mathcal{N}(\mathbf{x}_T; \mathbf{0}, \mathbf{I})$, and each reverse transition is modeled as a Gaussian whose mean and covariance are functions of a neural network:

$$p_{\theta}(\mathbf{x}_{t-1} | \mathbf{x}_t) = \mathcal{N}(\mathbf{x}_{t-1}; \mu_{\theta}(\mathbf{x}_t, t), \Sigma_{\theta}(\mathbf{x}_t, t)).$$



Diffusion models (backward)

- ✓ instead of predicting $\mathbf{x}_{t-1}$ directly, the network is trained to predict the noise that was added at step t .
- ✓ Suppose the network $\epsilon_\theta(\mathbf{x}_t, t)$ approximates the noise ϵ used in the forward process. Then the mean μ_θ can be parameterized as

$$\mu_\theta(\mathbf{x}_t, t) = \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_\theta(\mathbf{x}_t, t) \right).$$

$$x_{t-1} = \mu_\theta(x_t, t) + \sqrt{\tilde{\beta}_t} z, \quad z \sim \mathcal{N}(0, I).$$

- ✓ At sampling time, the model generates a sample $\mathbf{x}_0$ by:

- Drawing $\mathbf{x}_T \sim \mathcal{N}(0, I)$
- For $t = T, T-1, \dots, 1$, sampling. $\mathbf{x}_{t-1} \sim p_\theta(\mathbf{x}_{t-1} \mid \mathbf{x}_t)$
- Returning x_0 as the generated sample.

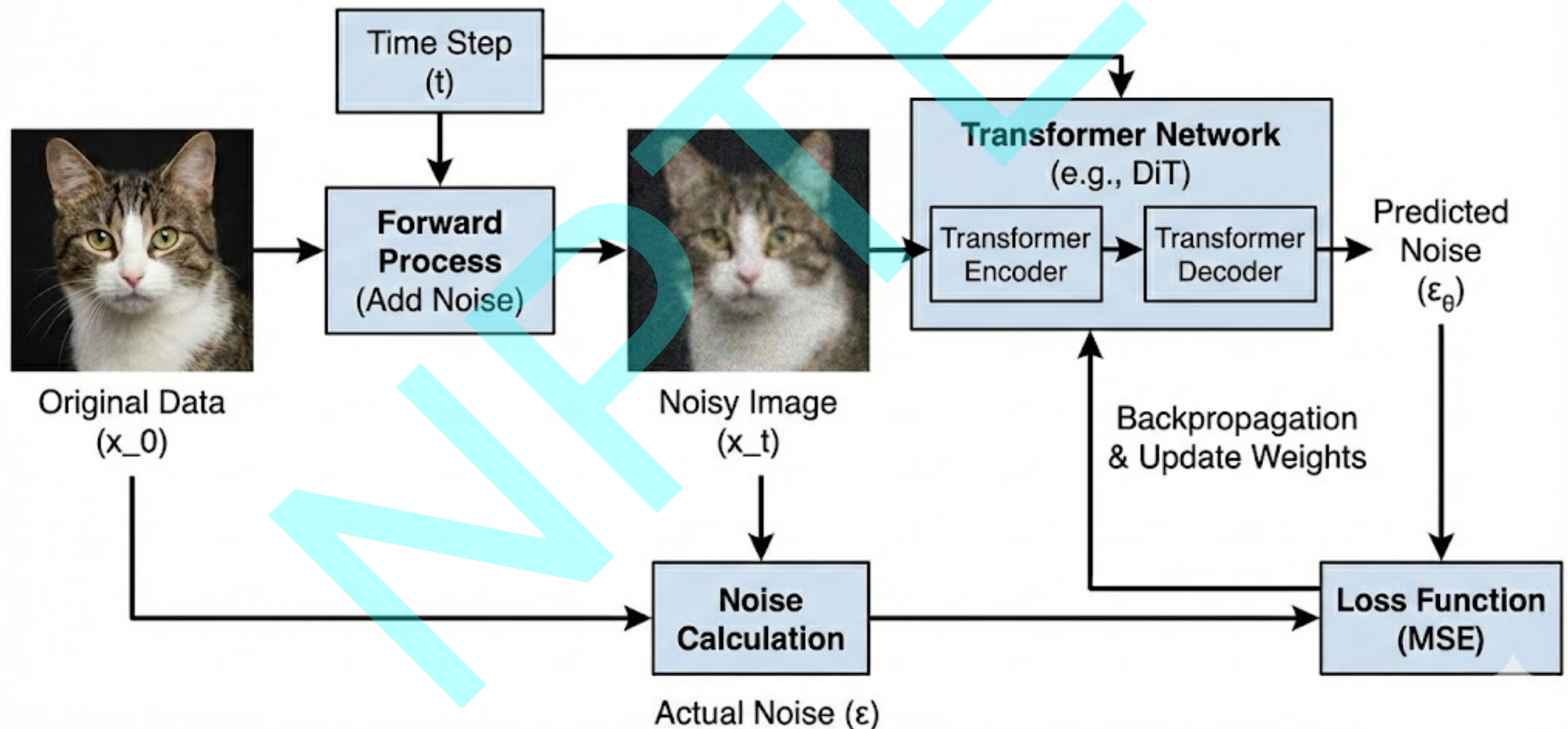
Diffusion model training

- ✓ Using the forward chain $q(x_{1:T} | x_0)$ as a variational distribution, we derive an evidence lower bound (ELBO) on $\log p_\theta(x_0)$
- ✓ After simplifications (and choosing a fixed variance schedule in the reverse model), the ELBO reduces to a very simple loss:

$$\mathcal{L}(\theta) = \mathbb{E}_{x_0, t, \epsilon} \left[\left\| \epsilon - \epsilon_\theta(x_t, t) \right\|^2 \right],$$

Diffusion Summary with Transformer

Diffusion Model Training Process

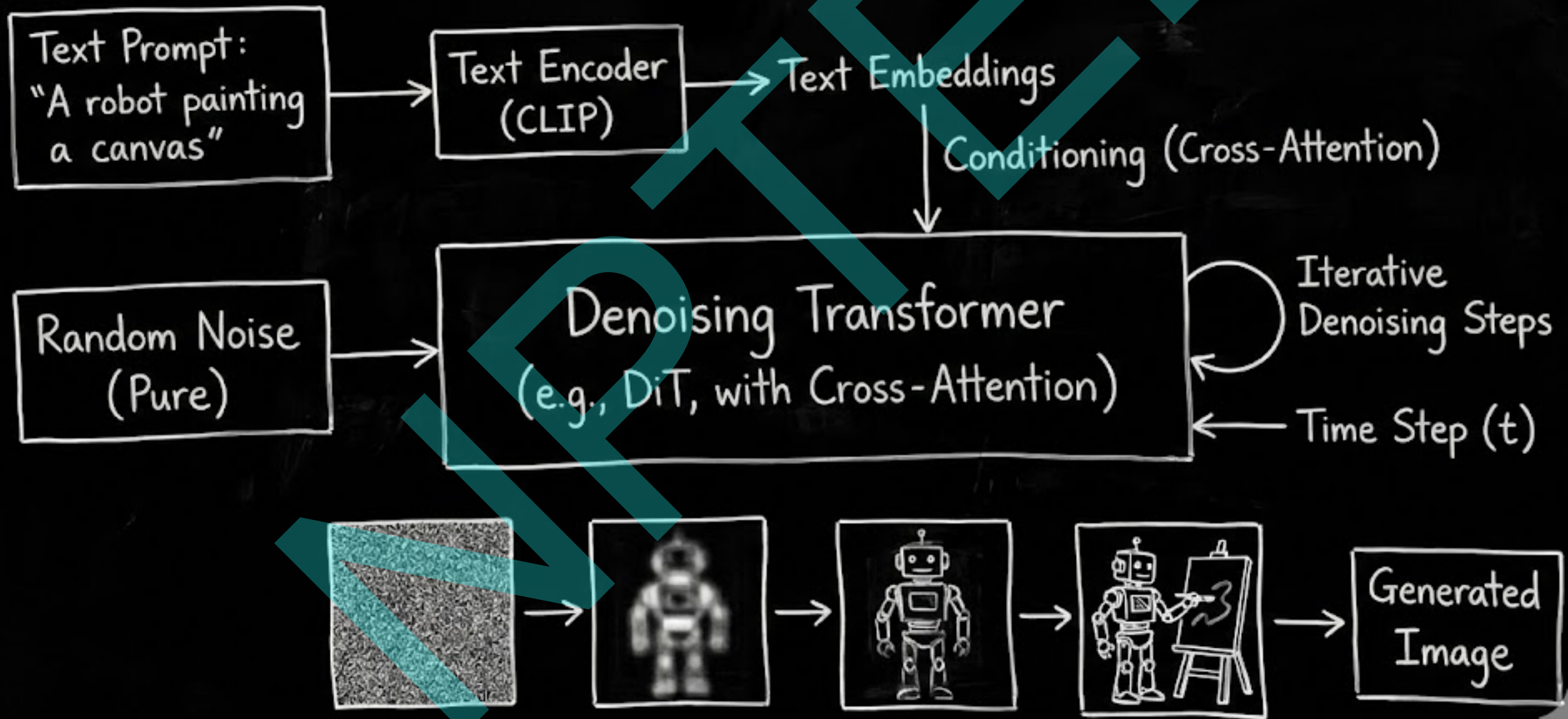


Applications of Diffusion

Domain	Example Uses
Images	Text-to-image generation, inpainting, style transfer
Video / 3D	Text-to-video, image-to-video, 3D asset generation
Audio & Speech	Text-to-speech, vocoders, music generation
Medical Imaging	Denoising, super-resolution, reconstruction
Synthetic Data	Data augmentation, domain adaptation, privacy-friendly samples
Robotics / Molecules	Trajectory planning, molecular & drug design

Text conditioned image generation

TEXT-CONDITIONED IMAGE GENERATION (DIFFUSION)



Latent Diffusion Models

High-Resolution Image Synthesis with Latent Diffusion Models

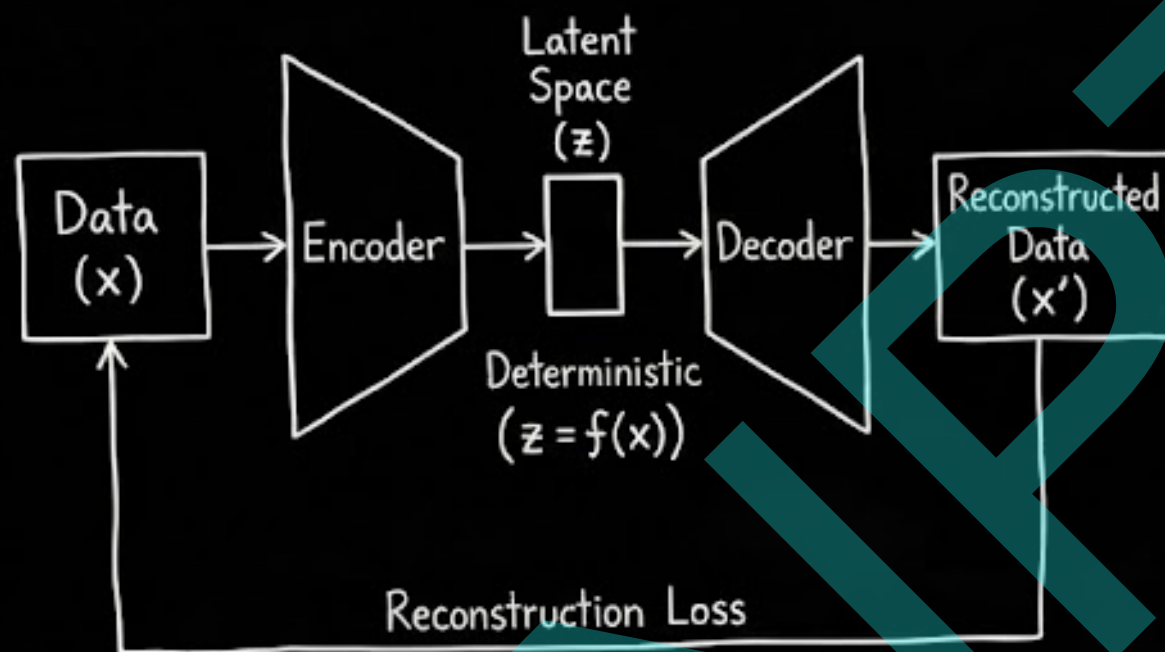
Robin Rombach¹ * Andreas Blattmann¹ * Dominik Lorenz¹ Patrick Esser^ℜ Björn Ommer¹

¹Ludwig Maximilian University of Munich & IWR, Heidelberg University, Germany ^ℜRunway ML

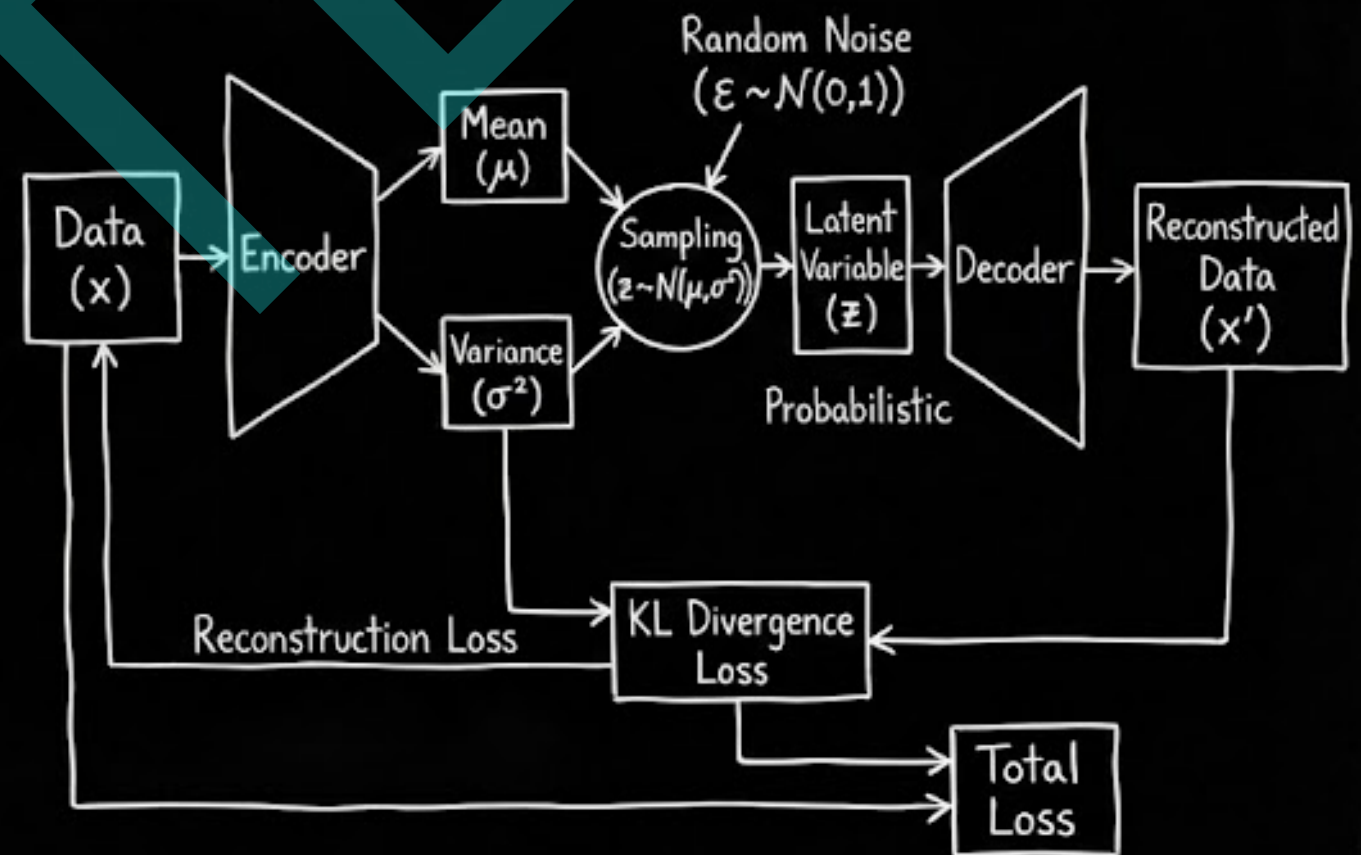
<https://github.com/CompVis/latent-diffusion>

3.4 Auto-encoder and Variational Autoencoder

AUTOENCODER (AE)

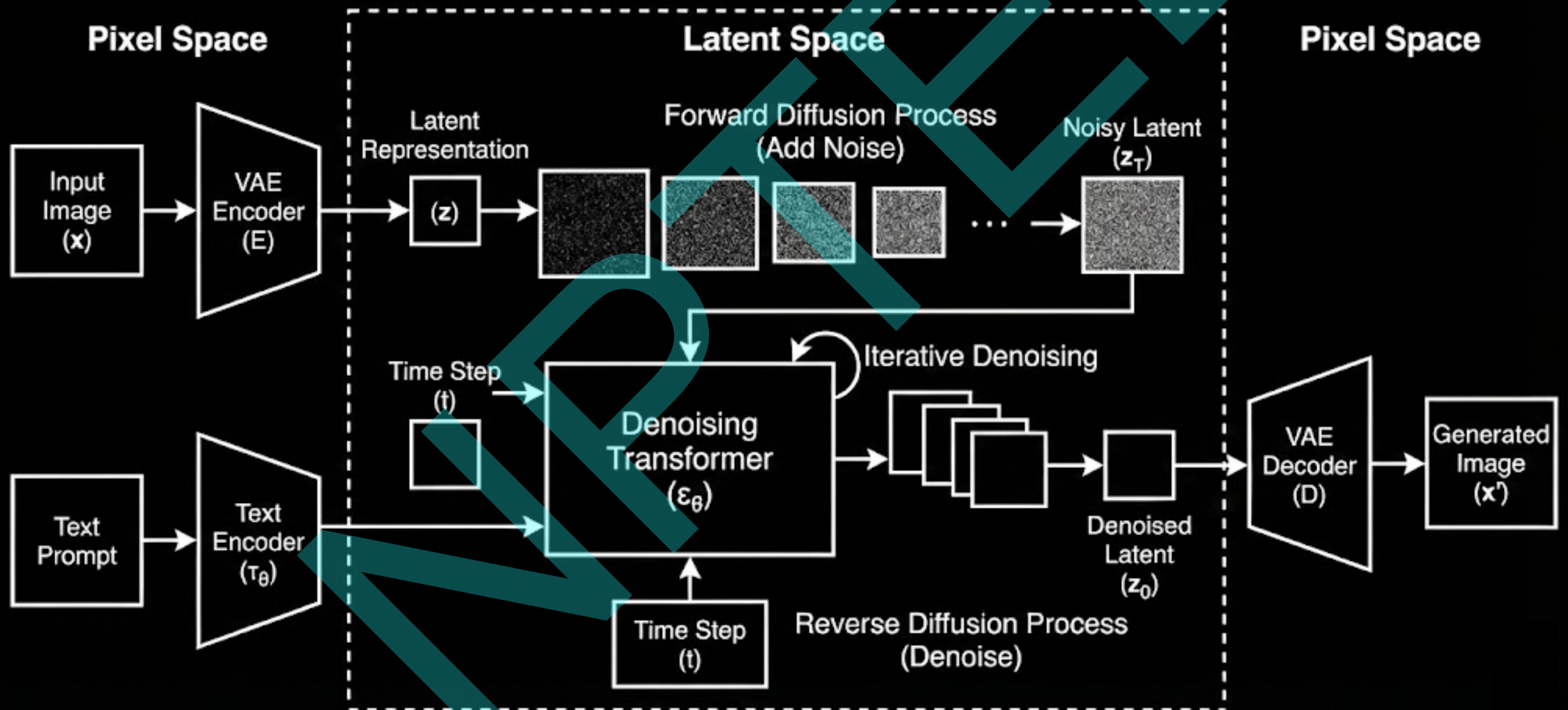


VARIATIONAL AUTOENCODER (VAE)



Stable Diffusion - Big Picture

STABLE DIFFUSION (Latent Diffusion Model)



Latent diffusion models - Step-by-step

Step 1

User starts with a text prompt

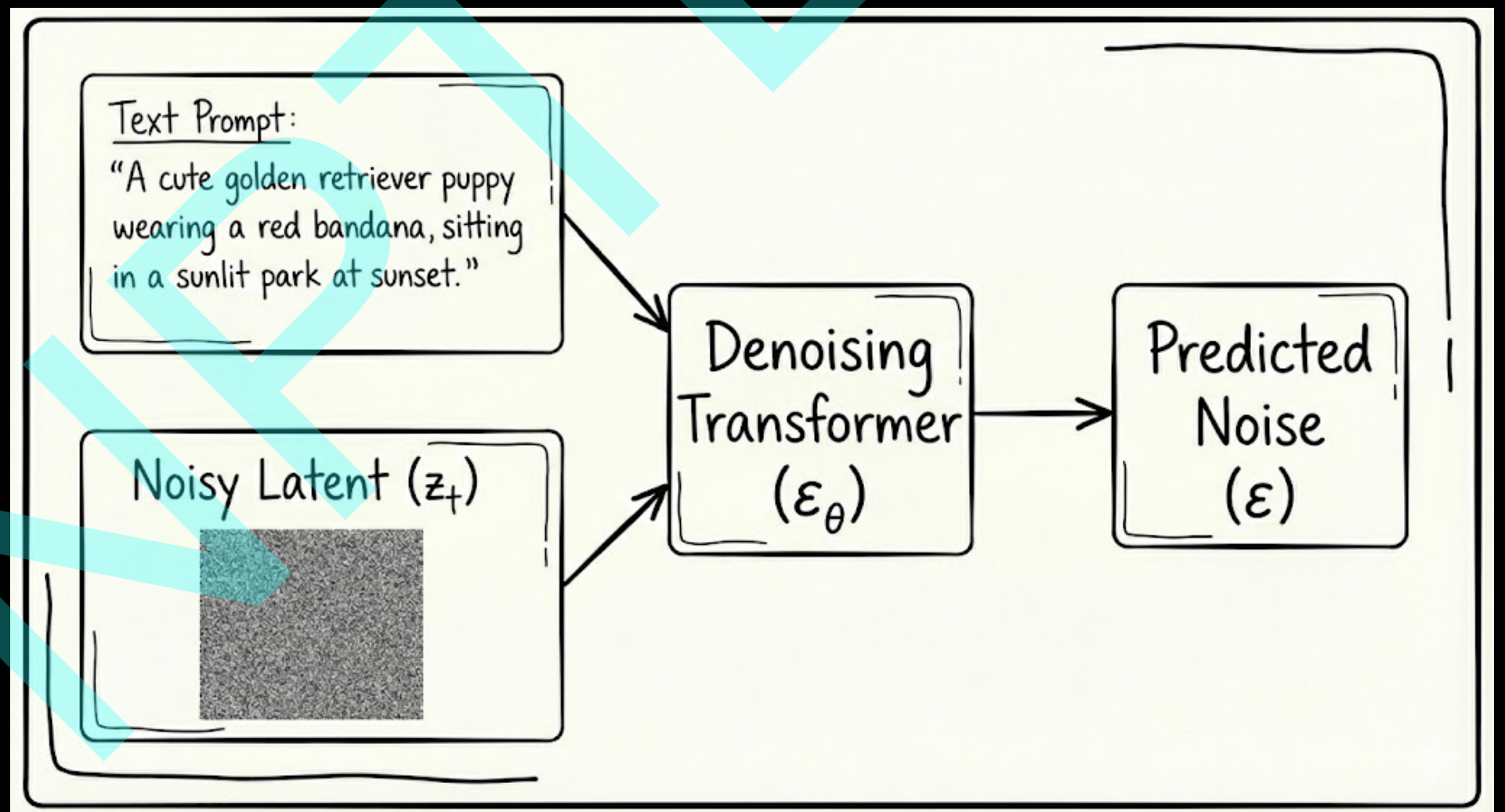
Text Prompt:

"A cute golden retriever puppy wearing a red bandana, sitting in a sunlit park at sunset."

Latent diffusion models - Step-by-step

Step 2

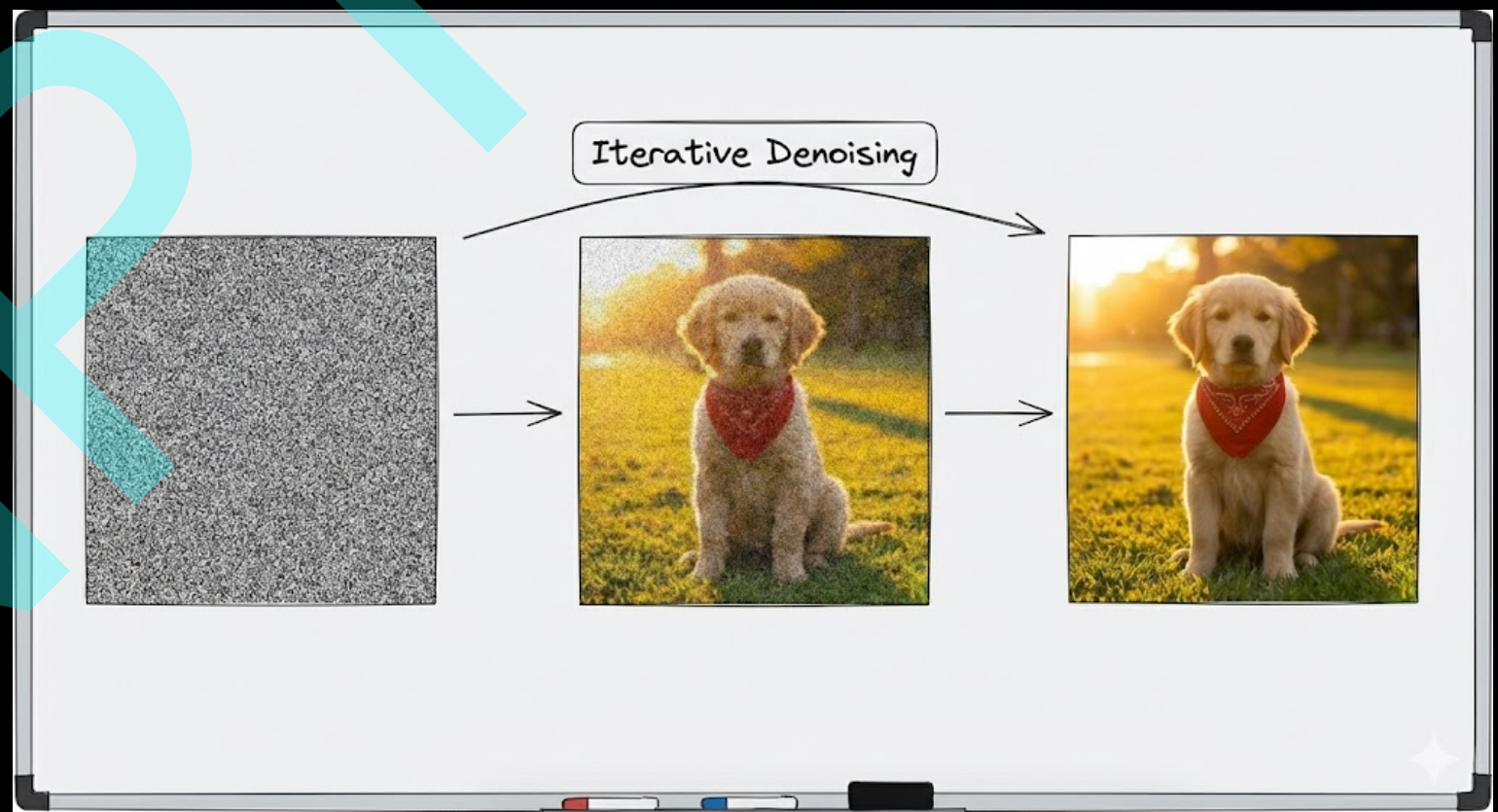
This prompt is fed into the Stable Diffusion model. Inside the model, a "Denoising Transformer" takes the text prompt (converted into numerical embeddings) and a noisy image (a "noisy latent") as input.



Latent diffusion models - Step-by-step

Step 3

The process from Step 2 is repeated over many steps. The model starts with pure noise and iteratively removes the predicted noise, guided by the text prompt. This gradually reveals the image, as shown in the sequence below.



Latent diffusion models - Step-by-step

Step 4

Complete decoding to generate top-quality image

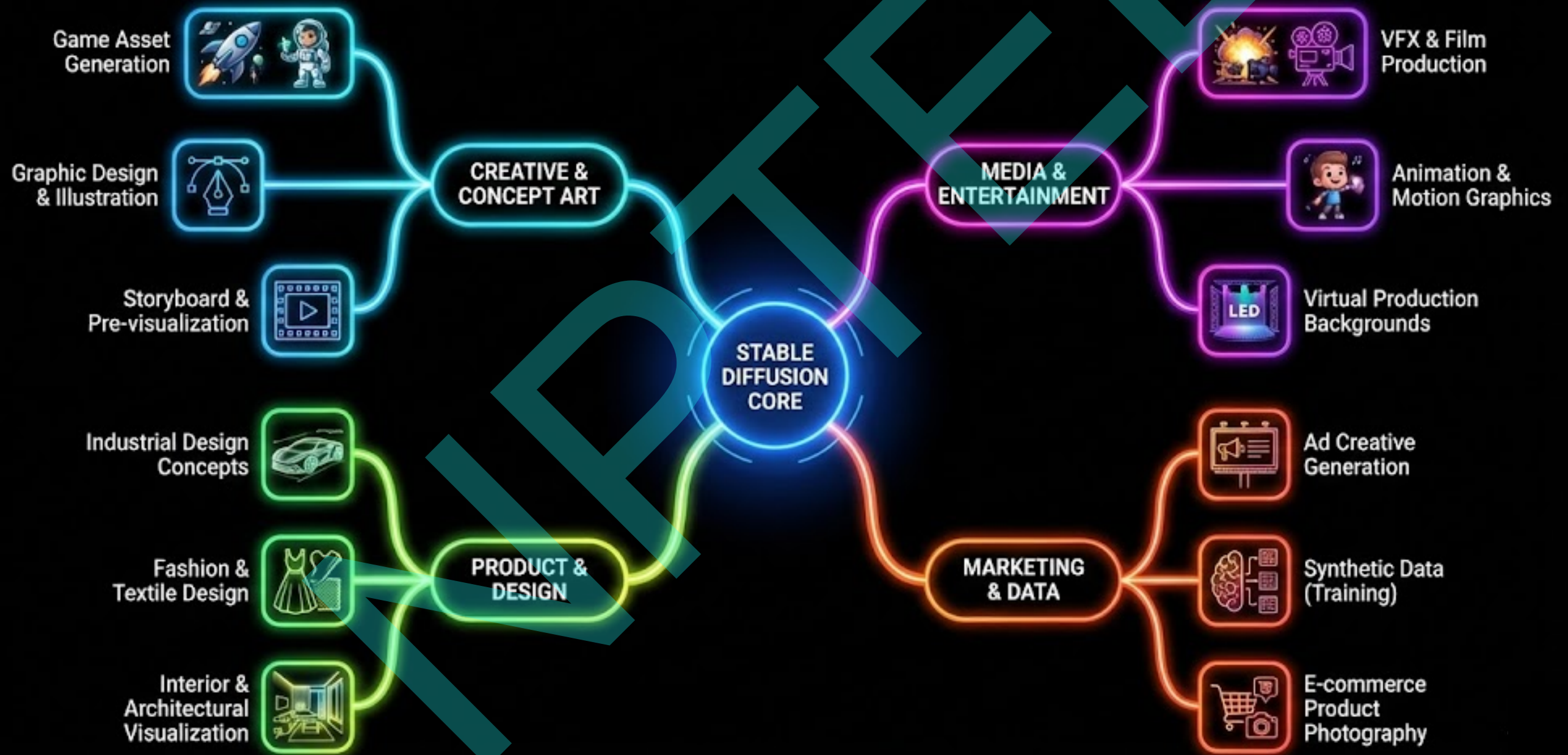


Diffusion versus Stable Diffusion

Aspect	Generic Diffusion Models	Stable Diffusion
What it is	Broad class of generative models using noise→denoise	A specific latent diffusion text-to-image model implementation
Space of diffusion	Operates directly in data space (e.g., pixels)	Operates in latent space of a VAE, then decoded to pixels
Conditioning	May be unconditional or conditioned (class, text, etc.)	Strong text-conditioned model via a text encoder (e.g., CLIP/T5)
Efficiency	Pixel-space diffusion is more compute & memory heavy	Latent-space diffusion is more efficient and faster for high-res images
Typical use case	General framework: images, audio, other modalities	Prompt-based text-to-image and image editing (inpainting, etc.)

Stable Diffusion - Applications

PRACTICAL APPLICATIONS OF STABLE DIFFUSION MODELS



Time for some ...

NPTFEL

Q&A

Contents - This Lecture

- ✓ 3.1 Large language models - test-time scaling
 - * Chain of Thought, Tree of thought, RAG
- ✓ 3.2 Model Compression
- ✓ 3.3 Diffusion
 - * Stable Diffusion
- ✓ 3.4 Benchmarking of LLMs
- ✓ 3.5 Reflections

Benchmarking LLMs

✓ Static dataset benchmarks (classic NLP style)

* Idea: Freeze a dataset → run the model on it → compute a metric.

✓ Examples: MMLU (multitask knowledge exams), BIG-Bench/BIG-Bench Hard, GSM8K (math), HumanEval/MBPP (code), TruthfulQA, SQuAD-style QA, etc.

✓ Typical metrics: accuracy, exact match (EM), BLEU/ROUGE, pass@k, perplexity etc

* Pros: Cheap, reproducible, easy to compare models.

* Cons: Quickly saturated, prone to test-set contamination, don't capture interactive behavior.

✓ Can also be extended large bench-mark suites (Example: OpenAI evals).

Benchmarking LLMs

✓ Human judgement

✓ Ask humans for pairwise comparisons (A vs B, which is better?),

- Likert scales (1–7 for helpfulness, correctness, style),

- * Used heavily for: chat quality, summarization, reasoning explanations, safety, UX.

- Pros: Captures nuance, real preferences, subtle quality differences.

- Cons: Expensive, slow, noisy; needs good guidelines and inter-rater checking.

- Track models over time and measure user preferences in the wild.

- Examples: Online A/B tests (which model do users pick?), <https://lmarena.ai/>

- Related - LLM-as-a-judge evaluation

Benchmarking LLMs

✓ Agentic style evaluation

- ✓ Idea: Evaluate models as agents performing multi-step tasks, often with tools.
- ✓ Examples (conceptually): web navigation tasks, coding agents fixing bugs, data analysis agents, multi-step reasoning with external tools or APIs.

*Metrics:

- Task success rate (did it complete the job?),
- Number of steps / tool calls,
- Cost, latency, error recovery.

*Pros: Closer to real-world usage; tests planning, memory, and tool use.

*Cons: More complex to set up; results depend on environment details (APIs, websites, sandboxes).

Benchmarking LLMs

✓ Stress-test models on harmful or adversarial inputs.

*Domains:

- Toxicity, hate, harassment.
- Jailbreak / prompt injection, tool misuse.
- Privacy leaks, data exfiltration.

*Approaches:

- Curated adversarial datasets,
- Automated attack generators,
- Human red-teaming exercises.

✓ Pros: Critical for deployment; exposes failure modes that accuracy benchmarks miss.

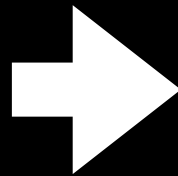
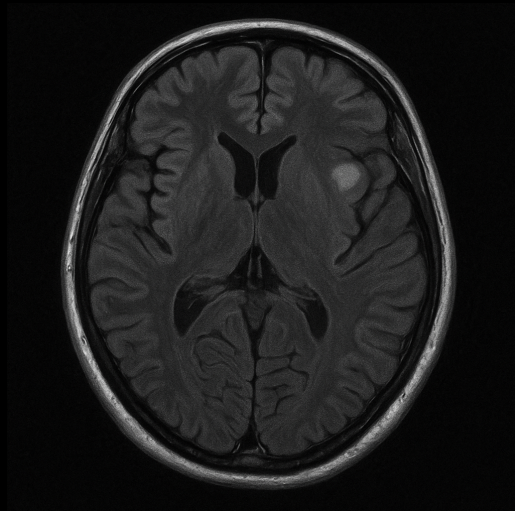
✓ Cons: Hard to be comprehensive; attackers evolve faster than static tests.

Contents - This Lecture

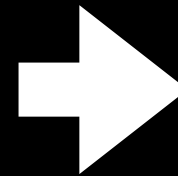
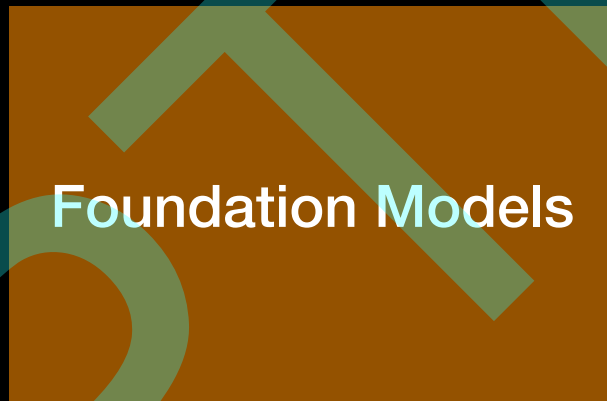
- ✓ 3.1 Large language models - test-time scaling
 - * Chain of Thought, Tree of thought, RAG
- ✓ 3.2 Model Compression
- ✓ 3.3 Diffusion
 - * Stable Diffusion
- ✓ 3.4 Benchmarking of LLMs
- ✓ **3.5 Reflections**

Reflection - Impact of Foundation Models

MRI



Foundation Models



Classification

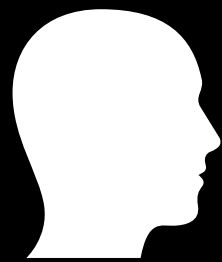
Healthy

Cancerous

Significant Impact on conventional ML applications

Reflection - Impact of Foundation Models

✓ Consuming multi-media



Traditional - Static



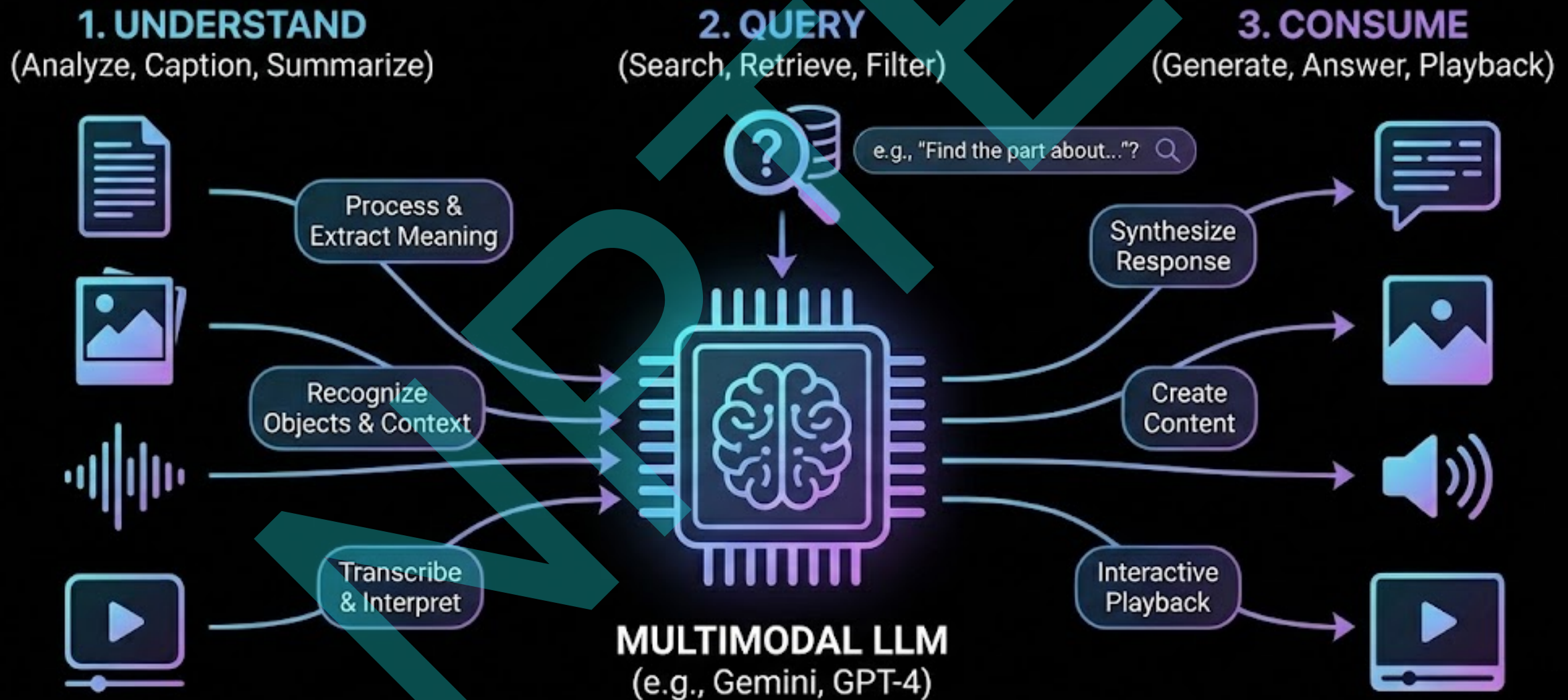
Prompt



Modern - Dynamic

Reflection - Summary

LLM POWERING MULTIMEDIA INTERACTION



Thank You

NOTES